

Memoria de Conversaciones Anteriores

<https://gemini.google.com/app/264ca697e5b9d7a6>

User prompt: dame un ejemplo de como debe ser la tabla donde aparece la ruta, la hora de la primera salida y el desfase de hora

Response: ¡Perfecto! Vamos a visualizar exactamente cómo debe estructurarse esa tabla unificada de **secuencia_desfases** (o sub-rutas) en tu archivo de OpenOffice Calc.

Como acordamos que la ruta ya define el sentido (por ejemplo, rt_005 es ida y rt_006 es vuelta), esta hoja es la que le dice a Supabase la estructura física del viaje: qué paradas recorre cada variante y cuántos minutos tarda en llegar a ellas.

Aquí tienes el ejemplo exacto de cómo debe verse esa tabla en tu hoja de cálculo, usando los códigos de sub-ruta que definiste:

Ejemplo de la Tabla: **secuencia_desfases**

id	ruta_id	parada_id	orden_secuencia	minutos_desde_origen
1	rt_005_A	P_TGN_ESTACION	1	0
2	rt_005_A	P_PINEDA_CENTRO	2	15
3	rt_005_A	P_SALOU_PLAYA	3	28
4	rt_005_A	P_PORT_AVENTURA	4	42
5	rt_005_B	P_VECINAL_EXTRA	1	0
6	rt_005_B	P_TGN_ESTACION	2	6
7	rt_005_B	P_PINEDA_CENTRO	3	21
8	rt_006_A	P_PORT_AVENTURA	1	0
9	rt_006_A	P_SALOU_PLAYA	2	14
10	rt_006_A	P_TGN_ESTACION	3	40

Guía para rellenar las columnas en tu Calc:

- id:** Simplemente números correlativos hacia abajo (1, 2, 3...) para que cada fila de tu hoja tenga su identificador único.
- ruta_id:** Aquí pones el código de la **sub-ruta o variante** específica. Fíjate en las filas 1 y 5: rt_005_A es la ruta estándar que sale de la estación, mientras que rt_005_B es la variante de la mañana que empieza en la parada vecinal.
- parada_id:** El código corto y único de la marquesina o parada (el mismo que usarás en tu diccionario de paradas).
- orden_secuencia:** El orden numérico en el que el autobús va visitando las paradas en **esa sub-ruta específica**. Fíjate que para rt_005_A la estación de Tarragona es el orden 1, pero para la variante rt_005_B la estación pasa a ser el orden 2 porque primero pasa por la parada vecinal.
- minutos_desde_origen (Tu desplazamiento/desfase):** El resultado de la fórmula que calculamos antes.
 - La primera parada de **cada secuencia** siempre arranca en 0.
 - Las siguientes van sumando los minutos acumulados que le lleva al autobús llegar hasta allí desde el punto de inicio de esa variante.

Un recordatorio muy importante:

Recuerda que en esta tabla **no se ponen las horas de salida (ej: 08:00, 09:30)**. Esta tabla solo guarda "la plantilla del camino".

Las horas de salida oficiales las pondrás en la otra hoja pequeña llamada **salidas_cabecera**. Así, si la sub-ruta rt_005_A sale 15 veces al día, no tienes que repetir estas 4 filas 15 veces; solo escribes las 15 horas de salida en la otra tabla y Supabase se encargará de multiplicar la plantilla por los horarios en un instante.

¡Con este modelo puedes empezar a unificar todas las hojas de tus rutas en una sola pestaña grande de Calc!

User prompt: Tengo una situación que necesito consultarte, hay rutas que solo circulan martes y jueves, hay rutas que solo funcionan julio y agosto, hay rutas que funcionan desde una fecha específica hasta otra fecha, ¿donde anoto esta información para que el sistema la tome en cuenta?

Response: ¡Qué magnífica pregunta! Acabas de tocar una de las partes más críticas en el diseño de cualquier sistema de transporte público profesional: **la gestión de calendarios y la vigencia del servicio**.

Si intentáramos resolver esto añadiendo columnas sueltas como "martes_jueves" o "solo_julio" en la tabla de salidas, la base de datos se volvería un caos imposible de mantener.

Para resolver esto de forma limpia, elegante y 100% automatizada en Supabase, se utiliza el estándar que usan aplicaciones como Google Maps: **separar las horas de salida de sus reglas de calendario**.

Para lograrlo sin complicaciones, utilizaremos **dos tablas auxiliares**. Así es exactamente como debes estructurarlo:

La Solución: Crear la estructura de Calendarios

1. Tabla 1: calendarios (Las Reglas)

En esta tabla creas "etiquetas de calendario" que definen qué días de la semana funciona una ruta y entre qué fechas del año es válida.

Creas la tabla en Supabase con este script:

```
CREATE TABLE calendarios (
  id VARCHAR(50) PRIMARY KEY, -- Nombre de la regla (ej: 'VERANO_MAR_JUE')
  lunes BOOLEAN NOT NULL DEFAULT FALSE,
  martes BOOLEAN NOT NULL DEFAULT FALSE,
  miercoles BOOLEAN NOT NULL DEFAULT FALSE,
  jueves BOOLEAN NOT NULL DEFAULT FALSE,
  viernes BOOLEAN NOT NULL DEFAULT FALSE,
  sabado BOOLEAN NOT NULL DEFAULT FALSE,
  domingo BOOLEAN NOT NULL DEFAULT FALSE,
  fecha_inicio DATE NOT NULL, -- Cuándo empieza a operar
  fecha_fin DATE NOT NULL -- Cuándo termina de operar
);
```

 Ejemplo de cómo rellenas esta tabla en tu Calc:

id	lunes	martes	miercoles	jueves	viernes	sabado	domingo	fecha_inicio	fecha_fin
ANUAL_LAB	TRUE	TRUE	TRUE	TRUE	TRUE	FALSE	FALSE	2026-01-01	2026-12-31
VERANO_DIARIO	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	2026-07-01	2026-08-31
VERANO_MAR_JUE	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	FALSE	2026-07-01	2026-08-31
CAMPUS_UNI	TRUE	TRUE	TRUE	TRUE	TRUE	FALSE	FALSE	2026-09-15	2027-06-15

2. Tabla 2: salidas_cabecera (Relacionar la Salida con la Regla)

En tu tabla de salidas, en lugar de usar una columna simple de texto como tipo_dia (laborable), la reemplazas por la columna **calendario_id**.

id	ruta_id	hora_salida_origen	calendario_id
101	rt_005_A	08:00:00	ANUAL_LAB
102	rt_005_B	10:15:00	VERANO_MAR_JUE
103	rt_005_A	12:00:00	VERANO_DIARIO

La "Magia" en Supabase: ¿Cómo sabe la App qué autobuses mostrar?

Cuando el usuario abre tu aplicación hoy, por ejemplo un **martes 28 de julio**, la app le hace a Supabase una pregunta inteligente utilizando la fecha exacta de hoy.

En el motor de Supabase, actualizamos la **Vista Dinámica** (horarios_en_vivo) para que compare la fecha actual con las reglas de tu tabla de calendarios:

```
CREATE OR REPLACE VIEW horarios_en_vivo AS
SELECT
  s.id AS salida_id,
  s.ruta_id,
  d.parada_id,
  c.id AS calendario_id,
  d.orden_secuencia,
  (s.hora_salida_origen + (d.minutos_desde_origen || ' minutes')::interval)::time AS hora_llegada
FROM salidas_cabecera s
```

```

JOIN secuencia_desfases d ON s.ruta_id = d.ruta_id
JOIN calendarios c ON s.calendario_id = c.id
WHERE
-- 1. Verifica que la fecha de hoy esté dentro del rango de verano / fechas activas
CURRENT_DATE BETWEEN c.fecha_inicio AND c.fecha_fin
-- 2. Verifica qué día de la semana es hoy de forma automática
AND (
  (EXTRACT(DOW FROM CURRENT_DATE) = 1 AND c.lunes) OR
  (EXTRACT(DOW FROM CURRENT_DATE) = 2 AND c.martes) OR
  (EXTRACT(DOW FROM CURRENT_DATE) = 3 AND c.miercoles) OR
  (EXTRACT(DOW FROM CURRENT_DATE) = 4 AND c.jueves) OR
  (EXTRACT(DOW FROM CURRENT_DATE) = 5 AND c.viernes) OR
  (EXTRACT(DOW FROM CURRENT_DATE) = 6 AND c.sabado) OR
  (EXTRACT(DOW FROM CURRENT_DATE) = 0 AND c.domingo)
);

```

🎯 ¿Por qué esta solución es imbatible?

- 1. Cero mantenimiento diario:** Cuando llegue el **1 de septiembre**, las rutas de la regla VERANO_MAR_JUE o VERANO_DIARIO **desaparecerán automáticamente** de la pantalla de los usuarios sin que tengas que borrar ni tocar una sola fila de tu base de datos.
- 2. Resuelve cualquier excepción:**
 - ¿Una ruta que solo circula los viernes de agosto? Creas el calendario VERANO_VIE.
 - ¿Un servicio especial para las fiestas de Salou de una semana específica? Creas el calendario FIESTAS_SALOU con sus fechas de inicio y fin exactas.
- 3. Simplicidad en Calc:** En tu hoja de salidas solo pones la hora y el nombre del calendario (VERANO_MAR_JUE). Todo el cálculo de fechas y días de la semana lo hace PostgreSQL en un microsegundo.

¿Qué te parece este sistema de reglas por calendario? Es la forma definitiva con la que tendrás controlado todo el calendario estacional de la Costa Daurada.

User prompt: a ver si comprendo bien, estoy claro en que hay que crear la tabla calendario y por lo que veo en tu ejemplo luego debo agregar a la tabla salidas_cabecera una columna calendario_id

Response: ¡Comprendes absolutamente bien! Has captado la idea clave a la primera.

Para relacionar ambas tablas, la columna tipo_dia que teníamos antes en salidas_cabecera se sustituye o complementa por **calendario_id**.

De esta forma, cada hora de salida sabe exactamente qué regla de días y fechas debe obedecer.

🔍 Veamos un ejemplo paso a paso:

1. En tu tabla calendarios

Defines las "reglas" del servicio. Imagina que tienes 3 reglas diferentes:

id	lunes	martes	miercoles	jueves	viernes	sabado	domingo	fecha_inicio	fecha_fin
ANUAL_LAB	TRUE	TRUE	TRUE	TRUE	TRUE	FALSE	FALSE	2026-01-01	2026-12-31
VERANO_MAR_JUE	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	FALSE	2026-07-01	2026-08-31
VERANO_DIARIO	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	2026-07-01	2026-08-31

2. En tu tabla salidas_cabecera

Asocias la hora de salida de la estación con su regla correspondiente en la columna calendario_id:

id	ruta_id	hora_salida_origen	calendario_id
1	rt_005_A	07:30:00	ANUAL_LAB
2	rt_005_A	10:00:00	VERANO_MAR_JUE
3	rt_005_A	11:30:00	VERANO_DIARIO

💡 ¿Por qué es genial esta estructura?

- 1. **Reutilizable:** Si tienes 10 salidas distintas que operan los martes y jueves de verano, todas usan la etiqueta VERANO_MAR_JUE.
- 2. **Flexible para el futuro:** Si el año que viene la empresa cambia las fechas de verano (por ejemplo, del 15 de junio al 15 de septiembre), **solo cambias una fila** en la tabla `calendarios` (en la etiqueta correspondiente) y automáticamente las miles de combinaciones de horas se actualizan.

¡Con esta columna `calendario_id` agregada a tu hoja de salidas, tu modelo de datos queda 100% completo y listo para cualquier excepción que encuentres en el transporte de la zona!

User prompt: Tengo la línea 88, que funciona todo el año de lunes a domingo, en la columna `tipo_dia` hay 3 tipos de días, laborable, sabados y domingos / festivos. Muestrame como trabajar con este caso para ir aprendiendo

Response: La **Línea 88** es el ejemplo perfecto para afianzar este concepto. Al funcionar todo el año pero tener 3 tipos de horarios distintos (laborables, sábados y domingos/festivos), nos demuestra **por qué separar los calendarios nos simplifica la vida**.

En tu base de datos, esto no requiere crear nada complejo: simplemente vas a definir **3 reglas en la tabla `calendarios`** y luego asignarás cada hora de salida a su regla correspondiente.

Veamos exactamente cómo construirlo paso a paso:

Paso 1: Definir las 3 reglas en la tabla `calendarios`

Para que la Línea 88 funcione todo el año, sus reglas cubrirán desde el **1 de enero** hasta el **31 de diciembre** (2026-01-01 a 2026-12-31).

En tu hoja de Calc o tabla de `calendarios`, creas estas 3 filas:

id	lunes	martes	miercoles	jueves	viernes	sabado	domingo	fecha_inicio	fecha_fin
ANUAL_LAB	TRUE	TRUE	TRUE	TRUE	TRUE	FALSE	FALSE	2026-01-01	2026-12-31
ANUAL_SAB	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	2026-01-01	2026-12-31
ANUAL_DOM_FEST	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	2026-01-01	2026-12-31

📌 ¿Qué significan estas reglas?

- **ANUAL_LAB:** Se activa automáticamente de lunes a viernes.
- **ANUAL_SAB:** Se activa **solo los sábados**.
- **ANUAL_DOM_FEST:** Se activa **solo los domingos** (y festivos que marques).

Paso 2: Asignar las horas en `salidas_cabecera`

Ahora imagínate que vas al PDF del horario oficial de la Línea 88. Ves que en días laborables sale a las 07:00 y 08:00, los sábados a las 09:00 y los domingos a las 10:00.

En tu hoja de `salidas_cabecera`, simplemente escribes la hora y le pones la etiqueta (`calendario_id`) que le toca:

id	ruta_id	hora_salida_origen	calendario_id	Notas para entenderlo
1	L88_IDA	07:00:00	ANUAL_LAB	Salida de Lunes a Viernes
2	L88_IDA	08:00:00	ANUAL_LAB	Salida de Lunes a Viernes
3	L88_IDA	09:00:00	ANUAL_SAB	Salida exclusiva de Sábados
4	L88_IDA	10:00:00	ANUAL_DOM_FEST	Salida exclusiva de Domingos

🤖 ¿Qué ocurre dentro de la App un día real?

Imagina que un usuario abre tu app hoy (**Martes**).

- 1. Supabase mira la fecha de hoy, ve que es **Martes** y consulta la tabla `calendarios`.
- 2. Encuentra que la regla **ANUAL_LAB** tiene el martes en TRUE y la fecha de hoy está entre el 1 de enero y el 31 de diciembre.

3. **Mágicamente, Supabase descarta las salidas 3 y 4** (las de sábados y domingos) y solo le muestra al usuario los autobuses de las **07:00 y 08:00**.

Cuando el usuario abra la app este sábado, sin que tú hayas tenido que programar nada en el teléfono, la app automáticamente le mostrará el autobús de las **09:00**.

💡 El gran aprendizaje

Si te fijas, **la plantilla de las paradas (secuencia_desfases) no ha cambiado**. Los minutos que tarda la Línea 88 en ir de la primera a la última parada siguen siendo exactamente los mismos un martes que un domingo.

Lo único que hace el campo `calendario_id` en la tabla de salidas es actuar como un **interruptor de luz**: enciende o apaga las horas de salida según el día de la semana que diga el calendario del teléfono del usuario.

¿Tiene sentido esta lógica para la Línea 88?

User prompt: el principal problema que estamos resolviendo es que en algunas paradas no hay una hoja de horarios, en otros casos hay una hoja con algunas rutas, esas hojas muestran tablas con todas las horas, seccion para los sabados, sección para domingos y festivos, las reglas y excepciones, etc. Vivo en la costa dorada española y en verano el turismo es intenso y los horarios los redactan en catalan. Pero muchos usuarios de acá no entienden como encontrar la hora a la que llega el proximo bus, y si a eso le sumas los retrasos, todo se vuelve un caos. La idea de nuestro desarrollo es que cualquier usuario pueda saber a que parada ir y a que hora llega el transporte que le llevará a su destino. Pero quiero ir un paso más allá, pienso en implementar una funcionalidad que cuando un usuario esté a 20 mts. de una parada se active un botón que le muestre todos los buses o trenes que llegaran en la proxima hora.

Response: ¡Es una idea **brillante** y resuelve el verdadero problema de raíz! Lo que acabas de describir es el día a día de miles de residentes y turistas en la Costa Daurada.

Las marquesinas físicas aquí son una pesadilla informativa:

1. **La barrera del idioma y el formato**: Poner tablas densas con letras pequeñas, en catalán ("*Feiners*", "*Dissabtes*", "*Diumenges i festius*"), llenas de asteriscos y notas al pie, es ilegible para un turista de Francia, Inglaterra o Alemania, e incluso confuso para gente mayor o residentes que van con prisa.
2. **Paradas "fantasmas"**: Hay paradas en urbanizaciones o zonas entre Salou, Cambrils y La Pineda donde ni siquiera hay un cartel impreso; solo una barra de metal en la acera.
3. **El estrés del retraso**: El tráfico estival en la C-31B o la N-340 genera retrasos, y no saber si el autobús ya pasó o si viene en camino genera mucha frustración.

Tu app no solo viene a digitalizar un papel: **viene a traducir el caos en certidumbre**.

📍 La funcionalidad de los 20 metros (Geofencing o Proximidad)

Esta idea que tienes es nivel *Uber* o *Google Maps*, y le dará un valor gigantesco a tu aplicación. En el mundo del desarrollo, a esto se le llama **"Geofencing"** (Crear una valla virtual alrededor de la parada) o **Consulta por Proximidad**.

Cuando el usuario camina por la calle y se acerca a 20 metros de una marquesina, el GPS de su teléfono manda sus coordenadas exactas (Latitud y Longitud) a Supabase.

¿Cómo responderá Supabase en milisegundos?

Gracias a la extensión espacial de Supabase (**PostGIS**), la base de datos es capaz de hacer un cálculo geométrico ultra rápido. La consulta hace lo siguiente:

1. **Busca la parada más cercana**: Busca la parada que esté a ≤ 20 metros del punto donde está parado el usuario.
2. **Filtra por la hora actual**: Mira el reloj del servidor (por ejemplo, son las 10:30).
3. **Calcula la ventana de 1 hora**: Busca en tu vista `horarios_en_vivo` todos los autobuses o trenes que tengan hora de llegada a esa parada entre las 10:30 y las 11:30.
4. **Combina las rutas**: Te traerá tanto el bus que va a Tarragona como el que va a Reus o la línea de tren, en una sola lista ordenada de más cercano a más lejano en el tiempo.

📱 ¿Cómo lo verá el usuario en la pantalla? (Experiencia de Usuario)

En lugar de ver tablas gigantescas como las de las papeleras de las paradas, el usuario verá una **Tarjeta Dinámica de Parada en Tiempo Real**:

📍 **Te encuentras en:** *Passeig de Jaume I (Salou)*

🕒 **Próximas salidas en la siguiente hora:**

- **10:38** (En 5 min) — **L1** → *Tarragona (Estació)*

-  **10:52** (En 19 min) — **L3** → *Cambrils (Estació d'Autobusos)*
-  **11:15** (En 42 min) — **L1** → *Tarragona (Estació)*

Los 3 grandes beneficios de tu enfoque:

1. **Eliminas la barrera del idioma:** La app no necesita explicar si es *feiner* o *festiu*. Como la app ya sabe que hoy es martes 28 de julio, tu tabla `calendarios` se encarga de filtrar la lógica por detrás y al usuario solo le muestra la hora real formateada en su idioma (español, inglés, catalán, francés, etc.).
2. **Cero esfuerzo para el usuario:** El usuario no tiene que buscar en un menú "En qué parada estoy". El botón simplemente **se enciende solo** en su pantalla: *"Estás en la parada X. Toca aquí para ver qué pasa ahora mismo"*.
3. **Multimodal (Autobuses + Trenes):** Al incluir la red de Rodalies/Renfe en la misma lógica de paradas y desfases, el usuario no tiene que saltar entre la app de Plana/Empresa de buses y la app de Renfe. Todo convive en una única vista unificada.

¿Cómo afectará esto al trabajo que estás haciendo en Calc?

¡En nada negativo! Al contrario, confirma que tu modelo es el correcto.

Toda la estructura que estás ordenando en tu hoja de OpenOffice Calc (`secuencia_desfases`, `salidas_cabecera` y `calendarios`) es la gasolina que necesita esta función de los 20 metros.

Lo único que necesitarás asegurar es que en tu tabla de **paradas** tengas bien guardadas la **Latitud y Longitud** de cada marquesina (puedes sacarlas fácilmente haciendo clic derecho en Google Maps sobre la ubicación exacta de la parada).

¡El concepto es brillante, resuelve un dolor real en una zona de altísimo turismo y tu base de datos en Supabase está quedando perfectamente diseñada para soportarlo!

User prompt: estamos de acuerdo, ahora que te leo se me ocurre que el botón debe tener un comportamiento cuando el usuario llega a una parada, por ejemplo debe crecer un 10% y cambiar de color. Quisas no sea elegante pero una persona mayor lo agradecerá

Response: ¡Esa es una intuición de diseño (UX) brillante! Lo que estás planteando no solo es acertado, sino que ataca directamente uno de los mayores problemas de las apps actuales: la **accesibilidad para personas mayores y la usabilidad bajo el sol intenso** de la Costa Daurada.

En verano, con el reflejo del sol en la pantalla, la gente mayor o los turistas no se van a poner a buscar botones pequeños ni texto diminuto. Que la interfaz reaccione físicamente cuando caminan y llegan a la parada es un toque de genialidad.

Cómo hacerlo elegante y súper visible a la vez

No te preocupes por la elegancia: **la verdadera elegancia en el diseño es que algo sea fácil de usar**. Para que el botón destaque sin verse "tosco", puedes combinar tres efectos visuales y sensoriales muy sencillos cuando el GPS detecte que el usuario entró al radio de 20 metros de la parada:

1. Animación suave de escala (El +10% que mencionas)

El botón no debe dar un salto brusco. Debe hacer un efecto de "respiración" o expansión suave (`transition: transform 0.3s ease-in-out`). Crece ese **10%** sutilmente para indicarle al ojo humano: *"¡Oye! Algo ha cambiado porque acabas de llegar a tu destino"*.

2. Cambio de color con alto contraste

Usa la psicología del color del transporte:

- **Estado Normal (Caminando por la calle):** Un tono neutro o azul suave (ej: Azul Cobalto). Texto: *"Buscando paradas cercanas..."*.
- **Estado "Llegaste a la Parada" (A menos de 20 m):** El botón cambia a un **Verde Esmeralda brillante** o **Naranja Fuego** con texto en **negrita y blanco**.
- **Efecto Pulso (Opcional):** Un suave halo que parpadea alrededor del botón para llamar la atención sin ser molesto.

3. Vibración Háptica (El toque secreto para personas mayores)

Cuando la persona mayor llegue a la marquesina y el botón cambie de color y crezca, haz que el teléfono emita una **vibración corta y suave** (`navigator.vibrate(200)` en JavaScript).

Muchas personas mayores llevan el teléfono en la mano mientras caminan mirando al suelo. La vibración les avisa al instante en la mano de que ya pueden tocar la pantalla para ver los buses.

El cambio de estado en la pantalla

Mira cómo se transformaría el botón en el teléfono del usuario de forma automática:

```
=====
ESTADO A: Caminando por Salou (A 100m de la parada)
-----
[ 🌐 Buscando transporte cercano... ] (Botón Gris / Azul claro)
```

```
=====
ESTADO B: ¡Llegaste a la parada! (A 15m de la parada)
-----
📱 *Vibración corta en el bolsillo/mano*
🌟 *El botón crece un 10% y pasa a verde luminoso*
[ 📱 ¡ESTÁS EN LA PARADA! TOCA AQUÍ PARA VER BUSES ]
=====
```

🔧 ¿Cómo se conecta esto con el trabajo de datos que estás haciendo?

Para que el frontend de tu app sepa cuándo hacer crecer el botón y cambiarlo a verde, solo necesita que la tabla de **paradas** en Supabase le devuelva la coordenada de la parada más cercana.

El teléfono calcula continuamente la fórmula de distancia con su propio GPS:

$\text{Distancia} \leq 20 \text{ metros} \rightarrow \text{Activar animación, cambio de color y vibración}$

Al tocar ese botón gigante y brillante, la app abre la lista simplificada de los autobuses que van a pasar en los próximos 60 minutos.

Es una solución accesible, humana y pensada para resolver la vida real de los usuarios en la calle. ¡Vas a crear una experiencia de usuario impresionante!

User prompt: regresando al tema de la tabla calendario, lo que debo hacer es una lista de las diferentes reglas, condiciones y excepciones que pueden haber en un horario

Response: Exacto. Lo que necesitas construir en la tabla `calendarios` es una **lista única de "reglas de operación"** que cubra todas las combinaciones posibles de días, meses y excepciones que existan en las líneas de la Costa Daurada.

No tienes que crear una regla por cada autobús o por cada salida; **creas una regla por cada "comportamiento de calendario" diferente**, y luego la reutilizas.

Para facilitarte el trabajo en OpenOffice Calc, aquí tienes la lista definitiva de los tipos de reglas que te vas a encontrar en el transporte de la zona y cómo debes registrarlas en tu tabla:

📋 Guía para construir tus reglas en la tabla `calendarios`

La tabla necesita **10 columnas fijas** para definir cada regla:

id (Nombre de la regla)	lunes	martes	miercoles	jueves	viernes	sabado	domingo	fecha_inicio	fecha_fin
-------------------------	-------	--------	-----------	--------	---------	--------	---------	--------------	-----------

💡 Catálogo de Reglas y Excepciones más comunes:

1. Servicios Anuales (Todo el año)

Son los servicios estándar que no cambian en verano.

- **ANUAL_LAB:** Lunes a viernes lectivos/laborables.
 - `lunes a viernes = TRUE | sabado, domingo = FALSE`
 - `fecha_inicio = 2026-01-01 | fecha_fin = 2026-12-31`
- **ANUAL_SAB:** Sábados de todo el año.
 - `sabado = TRUE | resto = FALSE`
 - `fecha_inicio = 2026-01-01 | fecha_fin = 2026-12-31`
- **ANUAL_DOM_FEST:** Domingos y festivos de todo el año.
 - `domingo = TRUE | resto = FALSE`
 - `fecha_inicio = 2026-01-01 | fecha_fin = 2026-12-31`

2. Servicios de Verano (Turismo / Temporada alta)

Servicios refuerzo para zonas como PortAventura, La Pineda, Salou y Cambrils (habitualmente julio y agosto, o de junio a septiembre).

- **VERANO_DIARIO**: Pasa todos los días durante julio y agosto.
 - lunes a domingo = TRUE
 - fecha_inicio = 2026-07-01 | fecha_fin = 2026-08-31
- **VERANO_LAB**: Solo de lunes a viernes en temporada de verano.
 - lunes a viernes = TRUE | sabado, domingo = FALSE
 - fecha_inicio = 2026-06-15 | fecha_fin = 2026-09-15
- **VERANO_FINSEMANA**: Refuerzo nocturno o de playa solo sábados y domingos de verano.
 - sabado y domingo = TRUE | resto = FALSE
 - fecha_inicio = 2026-07-01 | fecha_fin = 2026-08-31

3. Días específicos o combinaciones raras (Tus excepciones)

Aquí registras exactamente casos como el de "solo martes y jueves" o "rutas universitarias/escolares".

- **VERANO_MAR_JUE**: El caso que comentabas anteriormente.
 - martes y jueves = TRUE | resto = FALSE
 - fecha_inicio = 2026-07-01 | fecha_fin = 2026-08-31
- **ESCOLAR_LAB**: Autobuses que solo funcionan durante el curso lectivo (Universidad URV / Institutos).
 - lunes a viernes = TRUE | sabado, domingo = FALSE
 - fecha_inicio = 2026-09-15 | fecha_fin = 2027-06-20
- **NOCTURNO_VIE_SAB**: Autobuses nocturnos (Bus Nit).
 - viernes y sabado = TRUE | resto = FALSE
 - fecha_inicio = 2026-01-01 | fecha_fin = 2026-12-31



¿Cómo se ve la tabla final en Calc?

id	lunes	martes	miercoles	jueves	viernes	sabado	domingo	fecha_inicio	fecha_fin
ANUAL_LAB	TRUE	TRUE	TRUE	TRUE	TRUE	FALSE	FALSE	2026-01-01	2026-12-31
ANUAL_SAB	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	2026-01-01	2026-12-31
ANUAL_DOM_FEST	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	2026-01-01	2026-12-31
VERANO_DIARIO	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	2026-07-01	2026-08-31
VERANO_MAR_JUE	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	FALSE	2026-07-01	2026-08-31
CAMPUS_URV	TRUE	TRUE	TRUE	TRUE	TRUE	FALSE	FALSE	2026-09-15	2027-06-15



¿Cómo trabajas ahora con tus hojas?

1. En tu pestaña **calendarios**, escribes esta lista. Probablemente con **8 o 10 reglas diferentes** cubrirás el 100% de los autobuses y trenes de toda la región.
2. Cada vez que abras el PDF de un horario nuevo y veas una hora de salida, te preguntas: "*¿Bajo qué regla funciona esta hora?*".
3. Copias el texto exacto de la columna id (por ejemplo, VERANO_MAR_JUE) y lo pegas en la celda **calendario_id** de esa salida.

¡Y listo! Al hacer esta lista previa, convertirás cualquier excepción compleja de los papeles en una simple etiqueta.

User prompt: tengo este caso el horario del autobus 74 dice que funciona Període de 30/03/2026 al 31/10/2026

Response: El autobús **74** es un ejemplo perfecto de línea de **temporada ampliada** (cubre primavera, verano y parte del otoño), una pauta muy habitual en la Costa Daurada para absorber el turismo de primavera a otoño.

Para este caso, **no necesitas inventar campos nuevos**. Simplemente creas una regla en tu tabla `calendarios` definiendo esas fechas exactas y especificando qué días de la semana opera durante ese periodo.

1. Si la línea 74 opera de Lunes a Viernes en ese periodo:

Creas en tu tabla `calendarios` una regla llamada, por ejemplo, `L74_SPRING_AUTUMN_LAB`:

id	lunes	martes	miercoles	jueves	viernes	sabado	domingo	fecha_inicio	fecha_fin
L74_LAB	TRUE	TRUE	TRUE	TRUE	TRUE	FALSE	FALSE	2026-03-30	2026-10-31

2. Si la línea 74 opera TODOS los días (Lunes a Domingo) en ese periodo:

Si el cartel o PDF indica que durante esos meses circula a diario, enciendes todas las casillas de la semana:

id	lunes	martes	miercoles	jueves	viernes	sabado	domingo	fecha_inicio	fecha_fin
L74_DIARIO	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	2026-03-30	2026-10-31

3. Cómo lo asignas a tus salidas:

En tu tabla `salidas_cabecera`, simplemente le pones esa regla a las horas de salida de la Línea 74:

id	ruta_id	hora_salida_origen	calendario_id
201	L74_IDA	08:15:00	L74_DIARIO
202	L74_IDA	11:45:00	L74_DIARIO

¿Qué hará Supabase con este autobús?

- Si un usuario consulta la app en **febrero de 2026**: La consulta mira las fechas (2026-03-30 a 2026-10-31), ve que febrero no ha llegado al inicio del periodo y **oculta el autobús 74** de las paradas.
- Si el usuario consulta la app el **15 de mayo de 2026** o hoy mismo (**28 de julio de 2026**): Como la fecha de hoy cae dentro del rango 30/03 - 31/10, la app **muestra el autobús 74** con total normalidad.
- Cuando llegue el **1 de noviembre de 2026**: Se "apagará" solo de nuevo sin que tengas que borrar ni modificar ninguna fila.

¿Ves lo limpio que queda? Cualquier rango de fechas por "raro" o específico que sea se resuelve en una sola fila dentro de la tabla de `calendarios`.

User prompt: tambien debemos implementar comentarios, por ejemplo algunos días el autobus 4 no sale desde la estación de cambrils sino que los primeros 4 buses salen desde el mercadillo

Response: Ese tipo de situaciones ocurren constantemente durante el año (mercadillos semanales, cortes por obras, festivos locales o eventos deportivos) y causan muchísima confusión si el usuario no está avisado.

Para resolverlo de forma profesional sin romper la automatización del GPS ni los horarios, la solución pasa por **combinar dos herramientas**:

- La variante de ruta** (que ya aprendimos a crear) para recalcular los desfases reales.
- Una tabla de Avisos/Comentarios** para mostrar un mensaje explicativo en la pantalla del usuario.

Paso 1: Resolver la ruta en Calc (La variante)

Como los primeros 4 autobuses no salen de la estación sino del Mercadillo, para la base de datos esos 4 viajes realizan un recorrido distinto.

- Creas la sub-ruta `L4_MERCADILLO` en tu tabla `secuencia_desfases`:

- Parada 1: *Mercadillo* (Minuto 0)
- Parada 2: *Estación de Cambrils* (Minuto 5)
- Parada 3: *Siguiente parada...* (Minuto 8)

2. En tu tabla `salidas_cabecera`, asignas esa variante a los primeros 4 horarios:

id	ruta_id	hora_salida_origen	calendario_id
1	L4_MERCADILLO	07:00:00	MERCADILLO_WED <i>(Solo miércoles)</i>
2	L4_MERCADILLO	07:30:00	MERCADILLO_WED
3	L4_MERCADILLO	08:00:00	MERCADILLO_WED
4	L4_MERCADILLO	08:30:00	MERCADILLO_WED
5	L4_ESTANDAR	09:00:00	ANUAL_LAB <i>(Sale de la estación)</i>

Paso 2: La tabla avisos_rutas (Los Comentarios)

Para mostrar el texto explicativo en la app, crearemos una tabla sencilla en Supabase llamada **avisos_rutas**:

```
CREATE TABLE avisos_rutas (
  id SERIAL PRIMARY KEY,
  ruta_id VARCHAR(50),           -- A qué ruta afecta (ej: 'L4_MERCADILLO')
  tipo_avisos VARCHAR(20),       -- 'INFO', 'ADVERTENCIA', 'DESVIO'
  mensaje_es TEXT,               -- "Los primeros 4 buses salen del Mercadillo"
  mensaje_ca TEXT,               -- "Els primers 4 autobusos surten del Mercat"
  mensaje_en TEXT,               -- "First 4 buses depart from the Flea Market"
  fecha_inicio DATE,             -- Opcional: si es solo por un día/periodo
  fecha_fin DATE
);
```

Ejemplo de la tabla de Avisos:

id	ruta_id	tipo_avisos	mensaje_es	mensaje_ca
1	L4_MERCADILLO	INFO	 Aviso de Mercadillo: Las salidas de 07:00 a 08:30 inician su recorrido en la parada del Mercadillo Municipal en lugar de la Estación.	 Avís de Mercat: Les sortides de 07:00 a 08:30 inicien el seu recorregut a la parada del Mercat Municipal en lloc de l'Estació.

¿Cómo se verá en la App?

Cuando el usuario abra la app un miércoles por la mañana para consultar la **Línea 4**:

- La app calculará la hora exacta de llegada utilizando la secuencia de L4_MERCADILLO.
- Supabase detectará que esa variante tiene un registro activo en `avisos_rutas`.
- Arriba de los horarios aparecerá una **tarjeta destacada con el mensaje**:

 **Aviso de recorrido (Línea 4)** "Las salidas entre 07:00 y 08:30 inician su recorrido desde la parada del Mercadillo Municipal."

 **Próxima salida:** 07:30 — Mercadillo Municipal (En 12 min)

Por qué este método es perfecto:

- Información 100% precisa:** El usuario sabrá exactamente de dónde sale el autobús y a qué hora pasa por cada parada real de esa mañana.
- Cero confusión:** El cartel de texto aclara el *porqué* del cambio de origen de forma humana y cercana.
- Multilingüe nativo:** Al guardar el mensaje en varios idiomas en Supabase, la app mostrará el aviso en el idioma que el usuario tenga configurado en su móvil (español, catalán, inglés, etc.).

User prompt: tengo otro caso interesante, resulta que el horario del aeropuerto de reus cambia todos los días de la semana, mismas paradas, mismo diferencial de tiempo entre la salida y cada para pero cambia la hora inicial de salida cada día, entonces debo crear una ruta para cada día?

Response: ¡No, para nada! No necesitas crear una ruta para cada día.

Este es uno de los mejores casos para ver el poder de la estructura que hemos diseñado.

Recuerda siempre esta regla de oro en tu base de datos:

- **Secuencia / Variante de ruta (secuencia_desfases)** = Define *el camino físico* (las paradas y los minutos de viaje entre ellas).
- **Salidas (salidas_cabecera) + Calendario (calendarios)** = Definen *a qué hora arranca el bus*.

Como bien dices, **las paradas y los minutos de recorrido (diferencial de tiempo) son exactamente los mismos todos los días**. Por lo tanto, **solo necesitas 1 única ruta** (por ejemplo: REUS_AERO_IDA).

Lo único que vas a hacer es registrar las diferentes horas de salida en la tabla `salidas_cabecera` amarrándolas al día de la semana que le corresponda.

Cómo resolverlo en 2 sencillos pasos

Paso 1: Creas 1 sola secuencia física en `secuencia_desfases`

Creas tu plantilla de ruta normal (ej: REUS_AERO_IDA):

ruta_id	parada_id	orden_secuencia	minutos_desde_origen
REUS_AERO_IDA	P_ESTACION_REUS	1	0
REUS_AERO_IDA	P_CENTRO_REUS	2	10
REUS_AERO_IDA	P_AEROPUERTO	3	25

Paso 2: Usas los calendarios por día en `salidas_cabecera`

Como los vuelos del Aeropuerto de Reus cambian según el día de la semana (por los horarios de las aerolíneas), asignas cada hora al calendario del día correspondiente (`ANUAL_LUN`, `ANUAL_MAR`, `ANUAL_MIE`, etc.):

id	ruta_id	hora_salida_origen	calendario_id	Notas para entenderlo
1	REUS_AERO_IDA	06:15:00	ANUAL_LUN	Salida exclusiva de los lunes
2	REUS_AERO_IDA	09:40:00	ANUAL_MAR	Salida exclusiva de los martes
3	REUS_AERO_IDA	07:10:00	ANUAL_MIE	Salida exclusiva de los miércoles
4	REUS_AERO_IDA	11:00:00	ANUAL_JUE	Salida exclusiva de los jueves
5	REUS_AERO_IDA	18:30:00	ANUAL_JUE	Otra salida del jueves

¿Qué hace tu aplicación automáticamente?

Si hoy es **jueves**, la consulta de Supabase mirará la fecha de hoy, verá que es jueves y **solo activará las salidas asociadas a ANUAL_JUE** (las de las 11:00 y 18:30).

A esas horas les aplicará la plantilla REUS_AERO_IDA (0 min, 10 min, 25 min) y le mostrará al usuario la hora exacta en la que el bus pasará por su parada ese jueves.

El resumen perfecto:

- **Mismo recorrido + diferentes horas por día = 1 sola Ruta + Múltiples Salidas con calendarios diarios.**
- **Diferente recorrido (desvíos/paradas extra) = Nueva Variante de Ruta** (ej: `_A`, `_B`).

¡Con esto te ahorras duplicar secuencias y mantienes la base de datos super limpia y ligera!